CS342L – Advanced Database Management System

ADVANCED SQL 4: PL/pgSQL AND INTEGRATING SQL AS A BACKEND WITH PYTHON (DJANGO)

Ghulam Ishaq Khan Institute of Engineering Sciences

CS342L – Advanced Database Management System Lab

Lab 5 – Advanced SQL 4: PL/pgSQL and Integrating SQL as a Backend with Python (Django)

Objective

The objective of this session is to connect PostgreSQL to a real application layer — using Django and its ORM to query the dvdrental database, and calling the PL/pgSQL functions and procedures built in Labs 3 and 4 directly from Python views — so the database logic from this course becomes the backend of an actual web application.

Instructions

- Open the handout/lab manual in front of a computer in the lab during the session, with the dvdrental database already restored and available.
- Practice each new command by completing the examples and exercises.
- Turn in the answers for all the exercise problems as your lab report.
- When answering problems, indicate the commands you entered and the output displayed.

Contents

From PL/pgSQL to a Real Application 4
Setting Up Django with PostgreSQL 4
Installing Django and psycopg2 4
Creating a Django Project and App 4
Configuring DATABASES in settings.py 4
Mapping Existing Tables with Django Models 5
Using inspectdb to Generate Models 5
Unmanaged Models (managed = False) 5
A Hand-Written Model Example 5
Querying with the Django ORM 6
Basic Lookups 6
Joins via select_related and prefetch_related 6
Aggregation with annotate 6
What the ORM Actually Generates 7
Running Raw SQL from Django 7
Model.objects.raw() 7
connection.cursor() for Full Control 7
Calling PL/pgSQL Functions from Django 8
Calling PL/pgSQL Procedures from Django 8
Building a Simple View 8
A Function-Based View Returning JSON 8
Wiring Up a URL 8
Django Admin for Quick Data Browsing 9
Putting It All Together 9
Exercise 10

From PL/pgSQL to a Real Application

Labs 3 and 4 moved logic into the database itself: functions, procedures, and triggers that run inside PostgreSQL regardless of what calls them. This lab connects that database to an actual application layer — Django, a Python web framework — so the functions, procedures, and constraints already built become the backend of a real web application instead of something only reachable from psql.

Two ways of talking to the database will show up side by side throughout this lab: Django's Object-Relational Mapper (ORM), which turns Python method calls into SQL automatically, and raw SQL sent directly through Django's database cursor — the only way to call a PL/pgSQL function or procedure, since the ORM has no concept of either.

Setting Up Django with PostgreSQL

Installing Django and psycopg2

psycopg2 is the PostgreSQL driver Django uses under the hood; Django itself is the framework:

```
pip install django psycopg2-binary
```

Code language: Shell Session (shell)

Creating a Django Project and App

A project is the overall site configuration; an app is one component of it — here, the part that deals with the rental data:

```
django-admin startproject dvdrental_site
cd dvdrental_site
python manage.py startapp rentals
```

Code language: Shell Session (shell)

> **Note:** Add 'rentals' to the INSTALLED_APPS list in dvdrental_site/settings.py so Django knows the app exists.

Configuring DATABASES in settings.py

By default Django assumes SQLite. Point it at the existing dvdrental database instead:

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'dvdrental',
        'USER': 'postgres',
        'PASSWORD': 'your_password',
        'HOST': 'localhost',
        'PORT': '5432',
    }
}
```

Code language: Python (python)

Verify the connection from the Django side before writing any models:

```
python manage.py dbshell
```

Code language: Shell Session (shell)

> **Note:** dbshell drops you into psql using the credentials from settings.py — if it connects, Django can reach the database; if it fails, fix the DATABASES block before continuing.

Mapping Existing Tables with Django Models

dvdrental already exists, with data already in it — Django is joining an existing database, not creating one from scratch. That changes the usual Django workflow: instead of writing models first and letting Django generate the tables, the tables come first and the models are written (or generated) to match them.

Using inspectdb to Generate Models

Django can read the existing schema and write a first draft of models.py automatically:

```
python manage.py inspectdb > rentals/models.py
```

Code language: Shell Session (shell)

> **Note:** inspectdb's output is a starting point, not a finished product — it names every relationship DO_NOTHING by default and rarely guesses the most natural Python names, so review what it generates before relying on it.

Unmanaged Models (managed = False)

Normally Django's migration system owns a table — it can alter or drop it. Since dvdrental's tables already exist and are shared with plain SQL work from earlier labs, every model here is marked managed = False, telling Django "read and write this table, but never try to change its structure."

A Hand-Written Model Example

A cleaned-up version of what inspectdb produces for customer, with sensible Python names and an explicit link to store:

```
from django.db import models

class Store(models.Model):
    store_id = models.AutoField(primary_key=True)

    class Meta:
        managed = False
        db_table = 'store'


class Customer(models.Model):
    customer_id = models.AutoField(primary_key=True)
    store       = models.ForeignKey(Store, on_delete=models.DO_NOTHING)
    first_name  = models.CharField(max_length=45)
    last_name   = models.CharField(max_length=45)
    email       = models.CharField(max_length=50, blank=True, null=True)
```

```
    active      = models.IntegerField()

    class Meta:
        managed = False
        db_table = 'customer'
```

Code language: Python (python)

Note: db_table points the model at dvdrental's actual lowercase table name — without it, Django would look for a table named rentals_customer instead of customer.

Querying with the Django ORM

Once a model exists, Django can build SQL from Python method calls instead of writing SQL directly.

Basic Lookups

```
>>> Customer.objects.get(customer_id=1)
>>> Customer.objects.filter(first_name__startswith='A')
>>> Customer.objects.exclude(active=0)
```

Code language: Python (python)

Each of these corresponds directly to a WHERE clause from Lab 2 — get() expects exactly one row, filter() returns a queryset, and the double-underscore in first_name__startswith selects a lookup type, here equivalent to LIKE 'A%'.

Joins via select_related and prefetch_related

select_related follows a ForeignKey using a SQL JOIN in a single query — the right choice for the “one related row” side of a relationship, such as a rental's customer:

```
>>> Rental.objects.select_related('customer').filter(customer__customer_id=1)
```

Code language: Python (python)

prefetch_related instead runs a second query and joins the results in Python — the right choice for the “many related rows” side, such as every rental a customer has made:

```
>>> Customer.objects.prefetch_related('rental_set').get(customer_id=1)
```

Code language: Python (python)

Aggregation with annotate

annotate attaches a computed value to each row of a queryset, the ORM equivalent of GROUP BY with an aggregate function from Lab 2:

```
from django.db.models import Sum

>>> (Payment.objects
...     .values('customer_id')
...     .annotate(total_paid=Sum('amount'))
```

```
...     .order_by('-total_paid')[:5])
```

Code language: Python (python)

What the ORM Actually Generates

Any queryset can show the SQL Django built for it, which is the fastest way to understand — or debug — what the ORM is doing:

```
>>> print(Customer.objects.filter(first_name__startswith='A').query)
```

Code language: Python (python)

Note: Reading the generated SQL is also the best way to notice an accidental N+1 query problem — many small queries where select_related or prefetch_related should have produced one or two.

Running Raw SQL from Django

The ORM covers ordinary CRUD and simple aggregates well, but it has no idea what a PL/pgSQL function or procedure is — calling one means stepping outside the ORM and sending SQL directly.

Model.objects.raw()

raw() runs any SELECT and maps each returned row onto a model instance, as long as the primary key column is included:

```
>>> Film.objects.raw('SELECT * FROM film WHERE rental_rate > %s', [4.99])
```

Code language: Python (python)

connection.cursor() for Full Control

For anything that is not a plain SELECT on one model — a function call, a procedure CALL, or a query spanning several tables with no matching model — connection.cursor() is the direct equivalent of a psql session, reached from Python:

```
from django.db import connection

def dictfetchall(cursor):
    columns = [col[0] for col in cursor.description]
    return [dict(zip(columns, row)) for row in cursor.fetchall()]

def get_customer_count():
    with connection.cursor() as cursor:
        cursor.execute('SELECT COUNT(*) FROM customer')
        return cursor.fetchone()[0]
```

Code language: Python (python)

Note: Always pass parameters as the second argument to execute(), as in the examples below, never by formatting them into the SQL string — cursor.execute() parameterizes them safely, exactly like the %s placeholders used throughout this course's PL/pgSQL functions, closing off SQL injection the same way.

Calling PL/pgSQL Functions from Django

top_customers() from Lab 3 is an ordinary table-valued function from the database's point of view — Django calls it exactly like any other query, through the cursor:

```
def get_top_customers(limit=5):
    with connection.cursor() as cursor:
        cursor.execute('SELECT * FROM top_customers(%s)', [limit])
        return dictfetchall(cursor)
```

Code language: Python (python)

Calling PL/pgSQL Procedures from Django

A procedure is invoked the same way, using CALL instead of SELECT. Because rent_film() from Lab 3 can RAISE EXCEPTION when no inventory is available, that exception surfaces in Python as a psycopg2 error — catch it to turn a database-level business rule into a clean application-level response instead of a crash:

```
import psycopg2

def rent_film(customer_id, film_id, staff_id):
    with connection.cursor() as cursor:
        try:
            cursor.execute(
                'CALL rent_film(%s, %s, %s)',
                [customer_id, film_id, staff_id]
            )
        except psycopg2.errors.RaiseException as exc:
            return {'success': False, 'error': str(exc)}
    return {'success': True}
```

Code language: Python (python)

Building a Simple View

A Function-Based View Returning JSON

A Django view is just a Python function that takes a request and returns a response — here, the top_customers helper wrapped as JSON:

```
# rentals/views.py
from django.http import JsonResponse
from .db import get_top_customers

def top_customers_view(request):
    limit = int(request.GET.get('limit', 5))
    data = get_top_customers(limit)
    return JsonResponse(data, safe=False)
```

Code language: Python (python)

Wiring Up a URL

```
# rentals/urls.py
from django.urls import path
from . import views

urlpatterns = [
    path('top-customers/', views.top_customers_view),
]
```

Code language: Python (python)

Then include that file from the project's main urls.py:

```
# dvdrental_site/urls.py
from django.urls import path, include

urlpatterns = [
    path('api/', include('rentals.urls')),
]
```

Code language: Python (python)

Note: With the development server running (python manage.py runserver), GET /api/top-customers/?limit=10 now returns JSON built from a PL/pgSQL function running inside PostgreSQL.

Django Admin for Quick Data Browsing

Django ships with a ready-made admin site that can browse and edit any registered model — useful for quickly checking data without writing a view or leaving the browser:

```
# rentals/admin.py
from django.contrib import admin
from .models import Customer, Film

@admin.register(Customer)
class CustomerAdmin(admin.ModelAdmin):
    list_display = ('customer_id', 'first_name', 'last_name', 'email', 'active')
    search_fields = ('first_name', 'last_name', 'email')

admin.site.register(Film)
```

Code language: Python (python)

```
python manage.py createsuperuser
python manage.py runserver
```

Code language: Shell Session (shell)

Note: The admin site appears at /admin/ once a superuser account exists — it works immediately on unmanaged models, exactly as it would on any other model.

Putting It All Together

A realistic endpoint typically combines an ORM read, a raw-SQL call into PL/pgSQL, and proper error handling for the exception that call might raise — exactly what rent_film() needs, since a customer renting a film is precisely the kind of write that belongs behind a POST request with validation, not a GET.

```python
# rentals/views.py
import json
import psycopg2
from django.http import JsonResponse
from django.views.decorators.csrf import csrf_exempt
from django.views.decorators.http import require_POST
from .models import Customer, Film
from .db import rent_film

@csrf_exempt
@require_POST
def rent_film_view(request):
    body = json.loads(request.body)
    customer_id = body.get('customer_id')
    film_id     = body.get('film_id')
    staff_id    = body.get('staff_id')

    if not Customer.objects.filter(customer_id=customer_id).exists():
        return JsonResponse({'error': 'unknown customer_id'}, status=404)
    if not Film.objects.filter(film_id=film_id).exists():
        return JsonResponse({'error': 'unknown film_id'}, status=404)

    result = rent_film(customer_id, film_id, staff_id)

    if not result['success']:
        return JsonResponse({'error': result['error']}, status=400)
    return JsonResponse({'status': 'rented'}, status=201)
```

Code language: Python (python)

The ORM handles the cheap existence checks that would otherwise need their own SQL; the PL/pgSQL procedure still owns the real business rule — inventory availability — so that rule is enforced no matter what calls rent_film(), Django included.

Exercise

These exercises build on the dvdrental_site project and the functions/procedures from Labs 3 and 4. Write each view in rentals/views.py, wire it up in rentals/urls.py, and record both the code and the JSON response your lab report.

1. Configure DATABASES in settings.py for the dvdrental database and confirm the connection by running python manage.py dbshell. Record the psql prompt it drops you into.

2. Run inspectdb, then hand-clean the generated Film and Category models: rename fields to sensible Python names, set managed = False, and set db_table correctly on each.

3. Write an ORM query that returns the 10 most recent rentals, each with its customer's name and film title attached via select_related, without triggering extra queries for either.

4. Write an ORM aggregation query equivalent to the GROUP BY + SUM exercise from Lab 2: total amount paid per customer, highest first.

5. Write a view classify_customer_view(request, customer_id) that calls the classify_customer() function from Lab 3 through the cursor and returns {"customer_id": ..., "classification": ...} as JSON.